

20-7 處理 Word 和 PDF 文件

本章開頭的〈20-2 RAG 系統的「資料準備與檢索」流程〉一節提到，LangChain 的「文件載入器 (DocumentLoader)」模組，提供讀取並轉換各種資料來源的功能，也就是說，原本處理、分割 Markdown 文件的程式，只要修改文件載入器部分，就能用於處理 Word 和 PDF 等文件。

請先在本單元的 Python 虛擬環境執行 pip 命令安裝處理 Word 文件的模組：

```
pip install unstructured[docx]
```

然後修改讀取、分割、存入詞向量資料庫的程式，把 MarkdownLoader (Markdown 文件載入器) 改成 WordDocumentLoader (Word 文件載入器)。

```
from langchain_community.document_loaders import (
    DirectoryLoader,
    UnstructuredWordDocumentLoader # 換成 Word 文件載入器
)
```

定義 DirectoryLoader 物件 (loader) 時，glob 參數要改成讀取所有 .docx 文件、loader_cls 參數改用 UnstructuredWordDocumentLoader。

```
loader = DirectoryLoader(
    dir_path,
    glob="**/*.docx", # 讀取所有 .docx 文件
    loader_cls=UnstructuredWordDocumentLoader
)
```

其餘部分不用改，完整的程式碼如下：

```

import os
import shutil
from langchain_community.document_loaders import (
    DirectoryLoader,
    UnstructuredMarkdownLoader )
from langchain_text_splitters import
RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings

dir_path = './docs/'          # 文件來源資料夾
db_path = "./chroma_db"       # 向量資料庫路徑
loader = DirectoryLoader(
    dir_path,
    glob="**/*.md",          # 讀取所有.docx文件
    loader_cls=UnstructuredMarkdownLoader
)
docs = loader.load()         # 載入文件

# 建立分割器然後分割文件
splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
                                           chunk_overlap=100)
chunks = splitter.split_documents(docs)

print(f"\n已將文件分割成{len(chunks)}個片段。")
if chunks:
    print("最後一個片段內容:\n", chunks[-1].page_content)
    print("中繼資料:", chunks[-1].metadata)
    print("內容字數:", len(chunks[-1].page_content))

# 建立嵌入模型
embedding_model = OllamaEmbeddings(model="nomic-embed-text")

if os.path.exists(db_path):
    print(f"發現舊的資料庫，正在刪除: {db_path}")
    shutil.rmtree(db_path) # 刪除整個資料夾

vector_store = Chroma.from_documents(
    documents=chunks,

```



```

        embedding=embedding_model,
        persist_directory=db_path # 指定儲存路徑
    )

    print(f"成功將 {len(chunks)} 個文本塊寫入 {db_path}")

```

把範例的 Word 文件 (neo_m1.docx) 存入虛擬環境的 docs 資料夾，再執行程式測試。文件處理完畢後，再執行讀取、解析使用者提問的 app.py，對它進行提問，結果跟之前的 Markdown 文件版本相同。

過濾中繼資料

PDF 文件載入器模組叫做 UnstructuredPDFLoader，使用之前也要在本單元的 Python 虛擬環境執行 pip 命令安裝處理 PDF 文件的模組：

```
pip install unstructured[pdf]
```

或者一次安裝處理 Markdown, Word 和 PDF 文件的模組：

```
pip install unstructured[md,docx,pdf]
```

請留意模組名稱之間的逗號前後沒有空格，或者，把模組名稱用引號包圍：

```
pip install "unstructured[md,docx,pdf]"
```

處理 PDF 文件的流程和使用 unstructured 模組處理 Markdown, Word 等文件一樣。但是，處理 Word 和 PDF 文件時要留意一點，unstructured 的設計目標是盡可能詳細、完整地從文件中提取資訊，包括檔案類型、頁碼、語言…等各種中繼資料。例如，一個文本區塊可能具有這樣的中繼資料：

```
{
  "source": "docs/neo_m1.docx",      # 檔案來源
  "page_number": 2,                  # 頁碼
  "languages": ["eng", "zho-Hant"]   # 內文的語言
}
```

但並非所有向量資料庫都能儲存各種型態的中繼資料，以 ChromaDB 為例，為了確保查詢的效率，ChromaDB 規定儲存在 metadata 字典中的資料值，只能是以下幾種簡單的資料類型：

- str（字串）
- int（整數）
- float（浮點數）
- bool（布林值）

除了上述之外的所有 Python 資料類型，都被視為不相容，包括：

- list（列表）
- dict（字典）
- tuple（元組）
- set（集合）

因此，把中繼資料存入 ChromaDB 資料庫之前必須先過濾掉不相容類型資料，像上面的 "language"（語言）的列表資料。底下是過濾後的樣子：

```
{
  "source": "docs/neo_m1.docx",      # 檔案來源
  "page_number": 2                    # 頁碼
}
```

LangChain 有提供過濾中繼資料的模組，請在程式開頭引用它：

```
from langchain_community.vectorstores.utils import (
    filter_complex_metadata)
```

處理 Markdown, Word 和 PDF 文件

底下是讀取、分割 Markdown, Word 和 PDF、存入詞向量資料庫的完整程式碼：

```
import os
import shutil
# 引用所有需要的載入器
from langchain_community.document_loaders import (
    DirectoryLoader,
    UnstructuredMarkdownLoader,
    UnstructuredWordDocumentLoader,
    UnstructuredPDFLoader
)
# 用於過濾掉複雜（不支援的）中繼資料
from langchain_community.vectorstores.utils import (
    filter_complex_metadata)
from langchain_text_splitters import
RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings

dir_path = './docs/'
db_path = "./chroma_db"

# 為每種文件類型建立一個 DirectoryLoader
# 載入 Markdown 檔案
loader_md = DirectoryLoader(
    dir_path,
    glob="**/*.md",
    loader_cls=UnstructuredMarkdownLoader
)
```



```

# 載入 Word 檔案
loader_docx = DirectoryLoader(
    dir_path,
    glob="**/*.docx",
    loader_cls=UnstructuredWordDocumentLoader,
    # show_progress=True,
    # use_multithreading=True
)

# 載入 PDF 檔案
loader_pdf = DirectoryLoader(
    dir_path,
    glob="**/*.pdf",
    loader_cls=UnstructuredPDFLoader
)

# 載入所有文件然後合併
print("正在載入 Word 文件...")
docs_docx = loader_docx.load()

print("\n正在載入 Markdown 文件...")
docs_md = loader_md.load()

print("\n正在載入 PDF 文件...")
docs_pdf = loader_pdf.load()

all_docs = docs_docx + docs_md + docs_pdf
print(f"\n成功載入 {len(all_docs)} 份文件。")

if not all_docs:
    print(f"在 '{dir_path}' 路徑下找不到任何支援的檔案 "
          "(.docx, .md, .pdf)。")
else:
    # 過濾掉複雜的中繼資料
    # 避免因 PDF 等文件的複雜中繼資料導致後續存入資料庫時出錯
    filtered_docs = filter_complex_metadata(all_docs)

```



```

# 建立分割器然後分割文件
splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
                                           chunk_
                                           overlap=200)

chunks = splitter.split_documents(filtered_docs)

print(f"\n已將所有文件分割成 {len(chunks)} 個片段。")
if chunks:
    last_chunk = chunks[-1] # 預覽最後一個片段的資訊
    source_file = os.path.basename(
        last_chunk.metadata.get('source', 'N/A'))
    print(f"最後一個片段來自檔案: {source_file}")
    print("內容預覽:\n",
          last_chunk.page_content[:200] + "...")
    print("中繼資料:", last_chunk.metadata)
    print("內容字數:", len(last_chunk.page_content))

# 嵌入與儲存向量資料庫
embedding_model = OllamaEmbeddings(model=
                                   "nomic-embed-text")

if os.path.exists(db_path):
    print(f"\n發現舊的資料庫，正在刪除: {db_path}")
    shutil.rmtree(db_path)

print("\n正在建立向量資料庫...")
vector_store = Chroma.from_documents(
    documents=chunks,
    embedding=embedding_model,
    persist_directory=db_path
)

print(f"\n成功將 {len(chunks)} 個文本塊寫入 {db_path}")

```